Neurocomputing 638 (2025) 130204

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Reward shaping in reinforcement learning for robotic hand manipulation

Zelin Deng [a], Yunlong Dong [b], Xing Liu [c,*]

[a] School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, China
[b] Department of Automation, Tsinghua University, China
[c] School of Mechanical Science and Engineering, Huazhong University of Science and Technology, China

ARTICLE INFO

Communicated by Y. Wang

Keywords:
Robotic hand manipulation
Reinforcement learning
Reward shaping

ABSTRACT

Reinforcement Learning (RL) enables robots to accomplish complex tasks. In the context of robotic hand manipulation, We present the design of an 8 Degree of Freedom (DoF) hand hardware platform and utilize PyBullet to construct a simulation environment for RL training. Robotic hand manipulation involves considering factors such as object models, specific tasks, and environmental interactions, making it a highly challenging problem. The advantage of RL is its ability to learn without requiring a model, relying solely on the definition of the reward function. While training RL demands significant computational time and engineering adjustments. To enhance the training efficiency and stability in RL, Reward Shaping is adopted to modify the reward function during training, resulting in improved training performance. When transferring the training policy to a real-world environment, it is necessary to address the differences between simulation and reality. This paper employs domain randomization to vary environmental parameters and robotic hand parameters, thus training a stable policy that can execute tasks in the real-world. With just 2 h of simulation training, we have successfully achieved the task of valve rotation in real-world scenarios, validating the effectiveness of the proposed approach in robotic hand manipulation.

1. Introduction

Reinforcement Learning (RL) has found extensive applications across various tasks in robotics [1–3]. Robotic hands are designed to mimic and partially mechanize the complex articulations of the human hand to manipulate a wide range of objects [4]. Their purpose is to assist or even replace human hands in specific tasks. However, despite the advancements in robotics, robotic hand manipulation remains a highly challenging problem. This complexity arises from various factors, including the need to understand object models, execute specific tasks, and interact effectively with the environment. In contrast to traditional approaches that rely on precise object models and predefined algorithms, RL presents a promising alternative. RL has the advantage of being model-free, meaning it does not require an exact model of the system. Instead, it relies on defining a reward function that guides the learning process. This characteristic makes RL particularly well-suited for addressing the complexities of robotic hand manipulation tasks.

Compared to the widely used industrial tools such as vacuum suction cups or parallel grippers [5], robotic hands offer a greater Degree of Freedom (DoF) and flexibility. Robotic hands excel in handling more complex and dynamic operating environments. And the biomimetic structural design of robotic hands [6] allows them to perform intricate functions, such as multi-point grasping and manipulation of objects. These hands can also assist individuals with disabilities in performing simple daily tasks [7]. Robotic hands achieve control and manipulation capabilities through the design of mechanical structures and transmission mechanisms, integrating multiple DoF in their articulated joints. They are equipped with sensors for measuring rotation, displacement, and force [8]. The design, integration, and control algorithms of robotic hands are all crucial aspects that influence their ultimate ability to flexibly manipulate objects.

With the development of electronic control systems and the improvement of integration, several designs of robotic hands with high integration and multiple DoF have emerged, such as the commercial ShadowHand [9] and a series of robotic hands developed by DLR [10] etc. In recent years, with the development of soft robotics technology, robotic hands designed based on soft materials [11,12] have better environmental interaction capabilities and compliant adaptability. However, there still exists a considerable gap in the control accuracy and response frequency of soft-material-based robotic hands compared to mechanical robotic hands. The design of mechanical robotic hands is mainly based on the imitation of human hands. However, under specific scenarios or requirements, mechanical design modifications can enhance adaptability to particular tasks. For example, [13] install a

* Corresponding author.
E-mail address: liuxing@hust.edu.cn (X. Liu).

https://doi.org/10.1016/j.neucom.2025.130204
Received 3 December 2024; Received in revised form 10 March 2025; Accepted 2 April 2025
Available online 10 April 2025
0925-2312/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

roller mechanism at the end of a mechanical finger to achieve complex rotation of objects at the end finger. Zuo et al. [14] design a parallel finger mechanism in order to achieve a more stable and compliant object grasping, which is driven and coordinated through gears. Such task-specific design modifications in robotic hands alleviate control challenges to some extent and enable better adaptation to specific tasks.

In fact, beyond the design of robotic hands, the control of these hands is equally vital and merits thorough research. The control problem of dexterous hands has long been a challenging research topic. Dexterous hands typically possess multiple degrees of freedom, with each joint requiring precise control, resulting in an extremely large state and action space. Additionally, tasks often involve delicate operations and complex physical interactions, such as grasping and rotating, which impose extremely high demands on control accuracy and real-time performance. Moreover, environmental uncertainties and sensor noise further exacerbate the difficulty of control. Han and Tedrake [15] propose a segmented affine fitting approach is utilized for localized trajectory stabilization in the robotic hand control, enabling the grasping of unstable objects. And Pfanne et al. [16] introduce the design of an impedance controller, facilitating the closure of multiple contact points for object grasping. Beyond tasks related to grasping, the ability to manipulate objects is of paramount importance for robotic hands. Chatzilygeroudis et al. [17] propose a benchmark set for deformable object manipulation, including scenarios such as attaching metal chips and fitting transmission belts, primarily targeting the assembly of deformable objects.

Manipulating objects with robotic hands is indeed a complex and variable process that goes beyond simple grasping. This complexity arises from various factors, including the number of contact points, the properties of the objects being manipulated, and the forces applied during manipulation [18]. Traditional control methods often face challenges when trying to provide accurate mathematical descriptions and quantifications for these complex manipulations. These methods may struggle to account for the intricate interplay of forces, object dynamics, and changing contact points that occur during manipulation tasks. Successful learning of control strategies for manipulating block objects is achieved through RL algorithms by modeling and simulating the interaction between the ShadowHand and block objects in a physics simulation engine [1]. However, disparities between the physics simulation engine and real-world environments may result in inaccuracies in policy learning. To address this issue, Allshire et al. [19] introduce domain randomization by adding random noise within the physics simulation engine, enabling RL to acquire more robust and real-world policies. Domain randomization [20] enhances the adaptability and robustness of control strategies learned through reinforcement learning by sampling simulation parameters from a distribution. This approach prevents the failure of real-world scenario transfer caused by inaccuracies in environmental parameter settings. Given that RL demands a substantial amount of interaction data for learning, Deng and Zhang [21] explore the use of different reward functions to enhance the learning efficiency of RL in robotic hand manipulation. [22] employ supervised learning through human demonstrations for object manipulation, although this method is limited to robotic hands with structures similar to the human hand, such as the ShadowHand. And Chen et al. [23] combine knowledge distillation and RL, enabling the policy network to acquire supplementary information and expedite the training of RL.

The analysis of the above literature reveals that RL can be effectively applied to tasks involving robotic hand manipulation. However, it also needs to overcome the challenges such as high computational demands and discrepancies between the physics simulation engine and real-world environments. These challenges impose greater demands on the learning efficiency and training stability of RL algorithms. The primary contributions of this paper are as follows:

1. We have designed a robotic hand with 8 DoF, capable of mimicking human actions to perform more complex and intricate tasks.
2. RL techniques are applied for controlling the robotic hand. Additionally, training is conducted in conjunction with reward shaping methods, resulting in a successful reduction in training time and an improvement in the final reward.
3. Through the use of domain randomization techniques, we have successfully transferred the policies trained in a simulated environment to real-world scenarios.

The remaining structure of the paper is as follows: Section 2 presents the structure of the robotic hand manipulation designed in this paper. In Section 3, the designed robotic hand manipulation is applied to the valve turning task, and its effectiveness is demonstrated through experiments. Finally, a summary of this paper is provided in Section 5.

2. Reward shaping in RL

RL can be conceptualized as a MDP [24]. A MDP is denoted as a tuple $(S, \mathcal{A}, T, R, \gamma)$, where $S, \mathcal{A}$ represent the sets of actions and states, respectively. The function $T(s'|s,a)$ represents the probability of transition from the state s to the state s'. The reward obtained when the action a is taken is denoted as $R(s,a,s')$. And the discount factor for the rewards is represented by γ. The policy, denoted as $\pi(a|s)$, is a mapping from the state to the action to be taken.

The objective of the RL algorithm is to obtain an optimal policy $\pi^*(a|s)$ that maximizes the cumulative sum of rewards $\sum_{t=0}^{\infty}\gamma^t R_t$. The value function is defined as follows:

$$V_\pi(s) \triangleq \mathbb{E}_{a_t\sim\pi(a_t|s_t)}[\gamma^t R_t|s_0 = s]. \tag{1}$$

And the action value function is:

$$Q_\pi(s,a) \triangleq \mathbb{E}_{a_t\sim\pi(a_t|s_t)}[\gamma^t R_t|s_0 = s, a_0 = a]. \tag{2}$$

Reward shaping entails modifying the reward function of a problem to expedite the training speed and efficiency of reinforcement learning algorithms [25]. This approach involves deliberately designing reward functions to guide the learning of an agent and encourage it to achieve desired behaviors more rapidly. Reward shaping enhances the efficiency of an agent's learning of target tasks by introducing additional reward signals. It addresses the issue of sparse rewards by providing dense feedback, enabling the agent to receive guidance at each time step. Simultaneously, it guides the agent to explore effective behaviors, avoiding futile attempts and accelerating policy convergence. Well-designed rewards also reduce the risk of falling into local optima, allowing the agent to adapt more quickly to complex task requirements, thereby significantly improving training precision. The concept of reward shaping can be expressed as:

$$R'_t \overset{\Delta}{=} R_t + \gamma\Phi(s_{t+1}) - \Phi(s_t), \tag{3}$$

here, $\Phi(s)$ denotes the potential function of state s. Accelerating training can be achieved when the initial value of $\Phi(s)$ is the optimal value function, denoted as $V^*(s) \triangleq \sup_\pi V_\pi(s)$. However, accurately estimating $V^*(s)$ can be challenging in a complex environment. Wiewiora et al. [26] extend the action a into the potential function, utilizing the action value function to represent the potential function, which can be summarized as:

$$R'_t \triangleq R_t + \gamma\Psi(s_{t+1}, a_{t+1}) - \Psi(s_t, a_t) \tag{4}$$

Since it is a challenging task to estimate the value of states accurately, LyapRL [27] suggests using R directly instead of $\Phi(s,a)$, then we can obtain:

$$R_t^{lyap} \triangleq R_t + \lambda(\gamma R_{t+1} - R_t), \tag{5}$$

where λ is the hyperparameter that balances the weights of the two components.

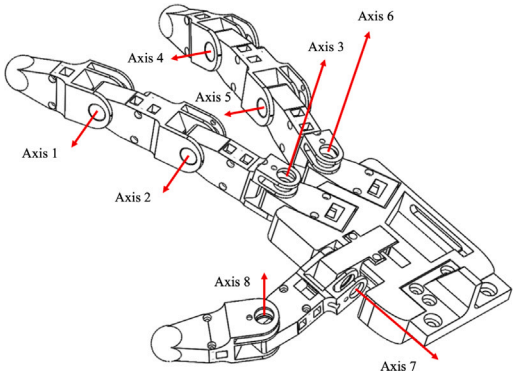


Fig. 1. The schematic diagram of D-Hand. D-Hand has 8 DoF within a compact design.



Fig. 2. The worm gear DC motor used in D-Hand. Worm gears can provide a significant reduction ratio in a very compact space, making them suitable for joint designs in robotic hands.

To mitigate the problem of reward drop during the training process, VAR [28] employs a variance reduction approach. The authors introduce a time smoothing method as follows:

$$R_t^{var} \triangleq \beta \hat{R}_{t-1} + (1-\beta)R_t, \tag{6}$$

where the hyper-parameter β is employed to balance the weighting of the rewards from two continuous time steps.

By combining LyapRL and VAR [28], the reward shaping used in this paper is as follows:

$$\begin{aligned} R_t^{lyap} &= R_t + \lambda(\gamma R_{t+1} - R_t), \\ R_t' &= \beta \hat{R}_{t-1}^{lyap} + (1-\beta)R_t^{lyap}, \end{aligned} \tag{7}$$

where the hyper-parameters λ and β can be adjusted to strike a balance between expediting training and improving stability. The goal of this paper is to integrate two advanced rewards shaping methods to speed up the RL training process and simultaneously improve the cumulative rewards achieved in the task. And the effectiveness of this method will be demonstrated through practical experiments.

3. Design of the robotic hand

This paper introduces D-Hand, an 8 DoF robotic hand comprised of three joint groups. Among these groups, the lateral finger joint group is specifically designed to mimic the human thumb, offering 2 DoF. The other two finger joint groups share a similar structural design, providing each of them with 3 DoF. Fig. 1 illustrates a schematic diagram of D-Hand's structure. It is evident that D-Hand, to a certain extent, replicates and simplifies the human hand's structure, featuring a total of 8 movable joints

The joints of a robotic hand need to achieve a wide range of motion within a confined space, which places high demands on the design of the joint motors. Therefore, D-Hand employs a lightweight and modular design, utilizing worm gear DC motors (shown in Fig. 2) with embedded magnetic encoders as drivers. The drive system utilizes a worm gear and worm structure, which offers a significant reduction ratio and a simple structure in a very compact space, leading to a substantial reduction in production costs and assembly complexity. By using the embedded magnetic encoders inside the motor, D-Hand can perform close-looped position control of 8 movable joints through RS485 and the specifics of the joints are shown in Table 1. The assembly diagram is shown in Fig. 3(b), the assembled prototype is shown in Fig. 3(c), and the simulated model in PyBullet [29] is shown in Fig. 3(d).

After constructing the prototype of D-Hand and developing the control software, it is crucial to evaluate D-Hand's object manipulation capabilities. The chosen test scenario involves having D-Hand grasp various objects. Multiple different objects are selected for testing, and D-Hand is controlled to perform grasping actions. Due to the varying shapes of the objects, D-Hand needs to adopt different grasping poses to successfully complete the object grasping tests. This grasping test helps assess D-Hand's joint mobility and its potential for object manipulation to a certain extent. The grasping results are shown in Fig. 4. It can be observed that our designed D-Hand effectively utilizes its 8 movable joints to adapt to various objects by employing different joint angles. This allows it to coordinate its movements and successfully grasp different types of objects.

Table 1
The details of noises n_s, n_a. It is worth noting that when sampling values from the normal distribution, they will be constrained within the specified range.

Joint	Range	Unit	Joint type
1	[0, 110]	Degree	Rotational
2	[0, 120]	Degree	Rotational
3	[−50, 50]	Degree	Rotational
4	[0, 110]	Degree	Rotational
5	[0, 120]	Degree	Rotational
6	[−50, 50]	Degree	Rotational
7	[0, 110]	Degree	Rotational
8	[0, 110]	Degree	Rotational

4. Valve rotation task

A valve rotation task has been constructed in this section. The robotic hand D-Hand is required to manipulate the valve through a series of movements involving its 8 rotational joints, achieving a continuous rotation process. Then we define the state s of the valve manipulation task. Firstly, the joint information of D-Hand, namely, the angular position information $\{j_i | i = 1, \ldots, 8\}$ of 8 joints, which uniquely determines the current status of the D-Hand. Simultaneously, the state s should also contain information about the operating object. This includes the distance between the hand and valve h, the angular position j_M, and velocity v_M. The state $s \in \mathbb{R}^{11}$ can be summarized by:

$$s \triangleq [j_1, \ldots, j_8, h, j_M, v_M] \tag{8}$$

After defining the state s, it is necessary to define the action a of this task. D-Hand has a total of 8 movable joints, so the action $a \in \mathbb{R}^8$ is defined as the position increment of each joint k at the next moment compared to the current moment:

$$a \triangleq [k_1, \ldots, k_8] \tag{9}$$

Because the goal of the valve rotation task scenario is to achieve continuous rotation of the valve, the reward function can be defined based on the rotation speed of the valve. In other words, the faster the valve rotates, the higher the reward value obtained. This setup effectively aligns with the continuous valve rotation requirement in the task scenario. Additionally, to discourage excessive joint movements by the D-Hand during object manipulation, a penalty term for incremental

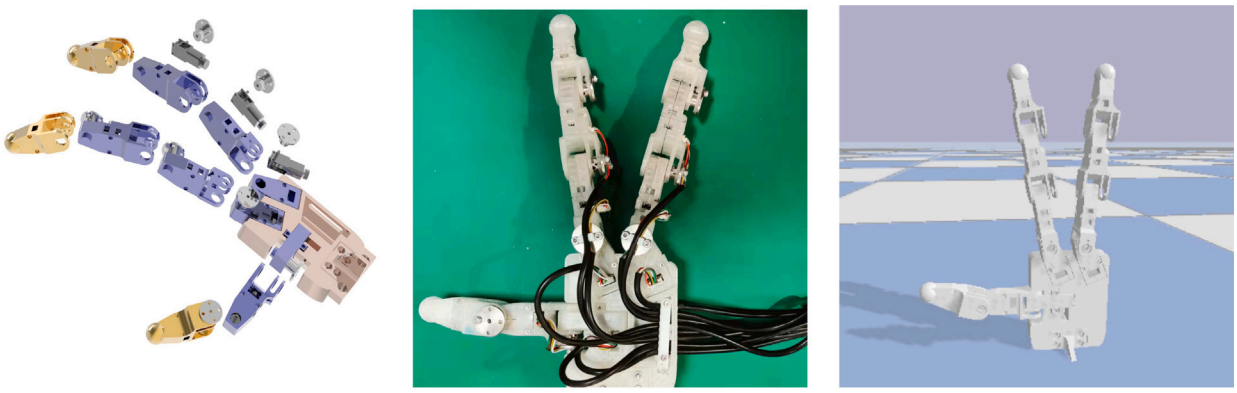


Fig. 3. The details of our designed 8 DoF robotic hand: D-Hand. (a) is the assembly diagram, (b) shows the assembled prototype, and (c) represents the simulation in PyBullet. D-Hand features a cost-effective, rapid assembly, and highly scalable design through structured joint design.



Fig. 4. D-Hand grasping illustrations for various objects. Experimental grasping trials with objects of different shapes validate D-Hand's manipulation capabilities over objects.

joint positions is incorporated into the reward function. In summary, the reward function can be defined as follows:

$$R = v_m - \mu\|a\|_2^2, \tag{10}$$

where $\mu \in \mathbb{R}^+$ serves as the weight coefficient for the penalty term, representing joint position increments.

After defining the states, actions, and reward function for the valve rotation task scenario, the training process begins. However, RL is highly inefficient in the early stages of training and can lead to many risky behaviors that could damage the robotic hand and the environment. Therefore, this paper chooses to initially train the policy in a simulated environment and then effectively transfer the policy to the real-world environment using appropriate algorithms. A PyBullet simulation environment for this scenario is set up in this section, as illustrated in Fig. 5. The one-to-one physical modeling of the D-Hand and the valve is carried out, making efforts to faithfully replicate properties such as mass and shape. The red part represents the manipulable object, which is the valve that can be continuously rotated. Above the valve is the D-Hand. The information about the D-Hand and the valve can be accessed through the PyBullet interfaces.

4.1. Simulated results

SD3 [30] algorithm is applied to train on the simulated valve rotation task. To address the challenge of the overestimation and underestimation bias in a continuous action space, SD3 uses a softmax operator defined as:

$$\text{softmax}_b Q(s,\bullet) \triangleq \int_{a\in\mathcal{A}} \frac{e^{bQ(s,a)}}{\int_{a'\in\mathcal{A}} e^{bQ(s,a')}da'}da, \tag{11}$$

where b is the controlling parameter. When $b \to \infty$, softmax degrades to $\max_{a\in\mathcal{A}} Q(s,a)$. SD3 applies two critic networks denoted as Q_1, Q_2 to form the TD target in training. Given the transition tuple (s_t, a_t, s_{t+1}, R_t), the TD target of SD3 is stated as:

$$y_t = R_t + \gamma \text{softmax}_b \min(Q_1(s_{t+1},\bullet), Q_2(s_{t+1},\bullet)) \tag{12}$$

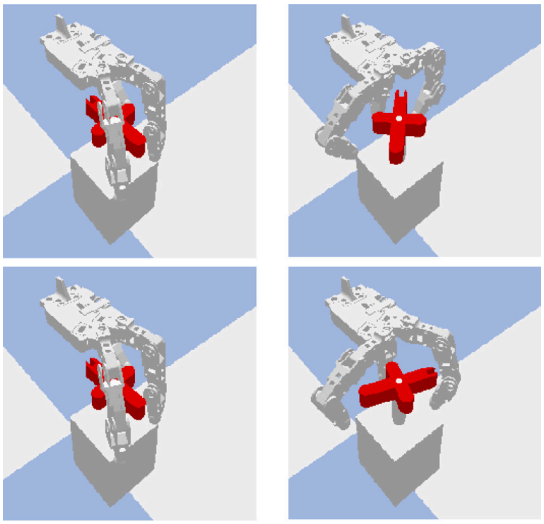


Fig. 5. The comparison results on the valve rotation task of whether using reward shaping. The first row and the second row represent the policy trained without reward shaping and with reward shaping, respectively.

Then the parameters of Q_1, Q_2 are updated by minimizing the Bellman loss :

$$\|y_t - Q_i(s_t, a_t; \theta_i)\|_2^2, \tag{13}$$

where $i = 1, 2$ and θ_i represent the trainable parameters of the two critic networks.

We set $b = 2$ during the training. The policy network used consists of the input layer, 1 hidden layer and 1 output layer. The input dimension is 11, the hidden dimension is 256, and the output dimension is 8. Both the input layer and the hidden layer use ReLU as the activation

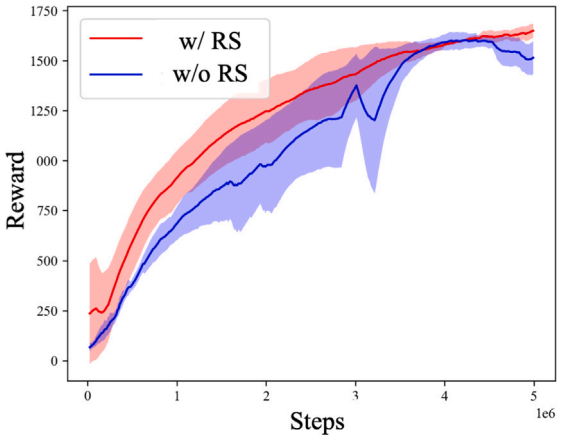


Fig. 6. The comparison of training results with (w/) and without (w/o) reward shaping (RS). The solid line and the shaded area represent the mean and variance across 10 repeated experiments on different random seeds.

function [31]. The output layer uses Tanh activation function. The discount factor γ is set to 0.99. The parameter μ in Eq. (10) is set to be 0.02 during the simulation. The computational environment used in the experiments consists of an Intel Core i5-11400 processor, a GTX 2080TI graphics card, and the PyTorch 1.8.1 framework for training. To enhance the training efficiency and stability, reward shaping proposed in Section 2 is applied. The hyper-parameters of the reward shaping are set as $\lambda = 0.01, \beta = 0.2$. The learning rate during the training process is set to 1e−4, and the maximum length of an episode is 600 time steps. The training process took about 2 h to complete 5 million steps during the simulation. For the hyperparameters λ, β, manual selection is required. Typically, the range for selecting these two parameters is $\lambda \in (0, 0.8), \beta \in (0, 2)$. However, the optimal values for these parameters need to be obtained experimentally for different tasks. We have designed an automatic optimization parameter selection method, which is detailed in [32].

The comparison training results of whether reward shaping is used are shown in Fig. 6. The experimental results are presented in the form of error bars to depict the mean and variance across 10 repetitions with different random seeds. It can be observed that the SD3 algorithm, when augmented with reward shaping, demonstrates superior experimental results compared to when reward shaping is not employed. This augmentation not only enhances the learning efficiency but also significantly improves training stability.

Furthermore, to provide a better visualization of the effectiveness of using reward shaping, we depict the rotation results of step 20 in Fig. 5. It can be seen that the policy trained with reward shaping can rotate the valve through a greater angle at step 20. This indicates that the rewards shaping method used in this paper is effective in significantly improving the training speed. A complete manipulation sequence of rotating the valve by 180 degrees is shown in Fig. 7. This demonstrates that, through the proposed RL algorithm, it is feasible to learn the control strategy for rotating the valve through interactive trial and error in a simulated environment.

4.2. Domain randomization

The ultimate goal of robotic hand manipulation is to apply these control policies in real-world environments. Therefore, we establish a real-world valve rotation environment, as shown in Fig. 8. The control policy can obtain joint angles in D-Hand through magnetic encoders. The rotation angle and velocity of the valve can be accessed from the magnetic encoder mounted on the valve's rotation axis. The distance h between the valve and the D-Hand is measured after installation.

After training a control policy in a simulated environment, effectively transferring this policy to a real-world environment is a key research challenge. This issue directly impacts whether the learned policy can be accurately and effectively applied in a real-world setting. Due to the inherent differences between a physical simulation engine and the real-world, learned policy in the simulation environment may be inaccurate or even ineffective in the real-world. For example, it is challenging for the physical simulation engine to accurately capture the features of the real-world environment, especially for complex tasks like manipulating objects, where factors such as nonlinear friction, elasticity, and dynamics are difficult to measure and model. To enhance the robustness and generalization of the policy learned in a simulated environment, this section employs domain randomization as a means to achieve effective transfer from simulation to reality.

Domain randomization can simultaneously address modeling errors in the environment and biases introduced by unavoidable noise. Its implementation involves randomly sampling within a certain range for critical parameters in the simulation environment, aiming to cover the parameter values that exist in the real-world environment. In this section, domain randomization operations are applied to the state observation values, the action values, and the environmental parameters in the valve rotation task. This can be represented as follows:

$$\begin{cases} \hat{s} = s + n_s \\ \hat{a} = a + n_a \\ \hat{E} = E \times n_e \end{cases}, \tag{14}$$

where $\hat{s}, \hat{a}$ represent the states and actions after domain randomization, and $\hat{E}$ indicate the values of certain key parameters (such as mass, friction coefficient, object scaling size, etc.). After domain randomization in the original environment, n_s and n_a represent the noise following a normal distribution $\mathcal{N}(\mu, \sigma^2)$, and n_e follows a uniform distribution. The details of n_s, n_a and n_e are shown in Tables 2 and 3.

The training process of RL based on domain randomization involves simulating and generating data in the simulation environment. During each simulation episode, environmental parameters are first sampled. Then, in each step of the simulation, corresponding domain randomization noise is added to both state observations and action values. Subsequently, we train RL over the noisy state observations and action values. These domain randomization noises serve to guide the RL algorithm in learning a more robust control policy to overcome critical sources of noise. The overall process is illustrated in Fig. 9.

Then we train SD3 with reward shaping and domain randomization. The training results are shown in Fig. 10. The inclusion of domain randomization noise in the environment has a certain impact on the training of RL. Initially, the introduction of noise leads to training instability. However, as the learning process continues, it becomes apparent that the learned policies can adapt and overcome the noise introduced by domain randomization, ultimately achieving greater robustness and suitability for transfer to a real-world environment. These experimental results also highlight the significant impact of environmental parameters on RL training, particularly in real-world applications of RL.

Then we apply the trained policy to the real-world valve rotation environment, and the manipulation results are shown in Fig. 11. The experiments show that the transferred control policy successfully achieves continuous rotation of the valve through coordinated movements of the 8 movable joints of the D-Hand. To verify the effectiveness of reward shaping and domain randomization, we conducted 20 repeated experiments involving valve rotation in the real-world. In each experiment, the initial valve angle is set to different values. If the policy succeeds in rotating the valve beyond 180 degrees within a span of 200 steps, the experiment is deemed a success; if not, it is classified as a failure. The ablation study results are shown in Table 4. It can be observed that both reward shaping and domain randomization play intuitively significant roles in transferring policies to real-world environments.

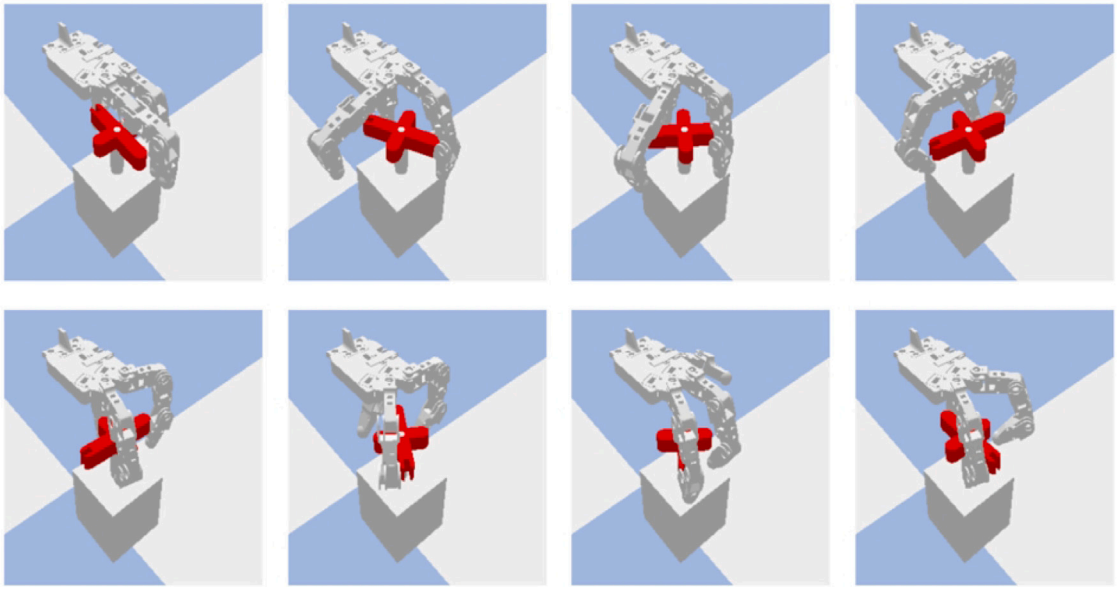


Fig. 7. Images of continuous valve rotation by D-Hand in the simulation environment (arranged sequentially from left to right, top to bottom). The trained policy demonstrates the ability to achieve valve rotation by utilizing the continuous motion of the 8 movable joints.

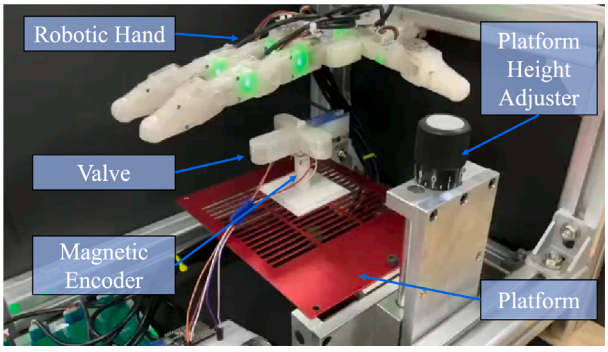


Fig. 8. The constructed real-world environment for valve rotation task. A magnetic encoder is mounted on the rotation axis of the valve to get the angle and the velocity. The Adjuster is used to adjust the height of platform.

Table 2
The details of noises n_s, n_a. It is worth noting that when sampling values from the normal distribution, they will be clipped within the specified range.

Parameters	Range	μ	σ	Unit
j_M	$[-0.03, 0.03]$	0	0.01	Rad
v_M	$[-0.08, 0.08]$	0	0.01	m/s
h	$[-0.05, 0.05]$	0	0.01	m
j_i	$[-0.06, 0.06]$	0	0.005	Rad
k_i	$[-0.02, 0.02]$	0	0.005	Rad

Table 3
The details of the uniform noise n_e. n_e will scale the parameters of the environments in physical simulation.

Parameters	Range
Mass of the valve	[0.96, 1.04]
Mass of the finger	[0.96, 1.04]
Size of the valve	[0.80, 1.20]
Friction coefficients	[0.70, 1.30]

5. Conclusions

In this paper, we have developed D-Hand, an 8 DoF robotic hand that integrates both hardware and software components. We establish a PyBullet simulation environment for RL training, avoiding the costly training expenses in a real-world environment. In order to enhance training efficiency and stability in RL, reward shaping is adopted to improve the training performance. Subsequently, we utilized domain randomization to transfer the trained control policies from the simulation environment to the real-world environments. This approach enabled the effective rotation of valves using D-Hand after just 2 h of simulation training. The RL training strategy based on domain randomization effectively bridges the gap between the simulation and real-world environments, allowing the acquisition of more robust control strategies for practical applications. The experiments underscored the significance of the proposed reward shaping framework in scaling up RL algorithms for real-world applications. In future work, we will further enhance the mechanical structure and control algorithms of D-Hand to tackle more complex manipulation tasks.

Table 4
The ablation study of our proposed method. 20 repeated experiments with different initial valve angles are conducted.

Reward shaping	Domain randomization	Success rate
No	No	30%
Yes	No	40%
No	Yes	65%
Yes	Yes	**80**%

CRediT authorship contribution statement

Zelin Deng: Writing – original draft, Validation, Methodology, Conceptualization. **Yunlong Dong:** Writing – review & editing, Visualization. **Xing Liu:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grants 52188102.

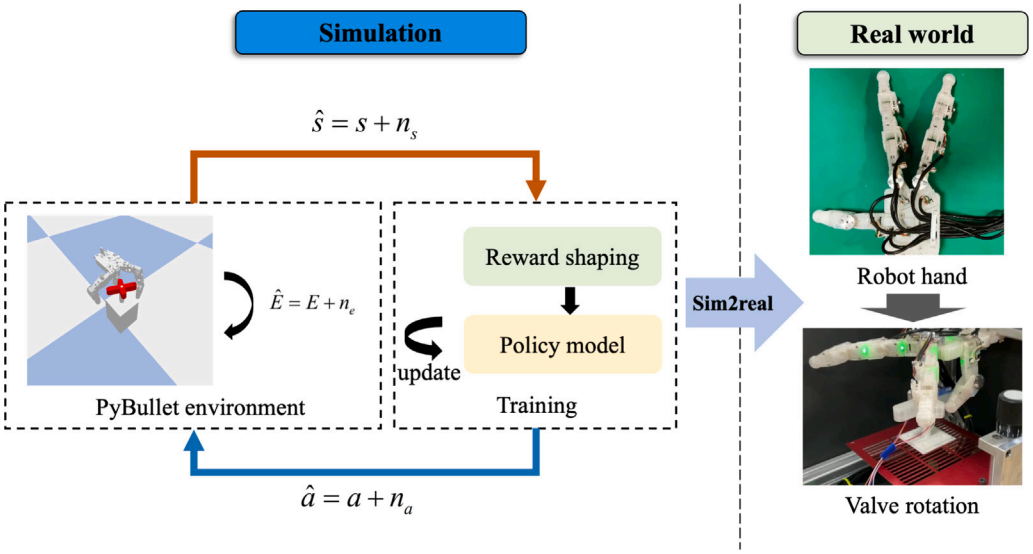


Fig. 9. The overall sim-to-real training process employed in this paper involves first learning the policy in a simulated environment and then transferring it to the real-world environment through domain randomization. The control frequency of the real dexterous hand is 1 kHz, while the control frequency of the reinforcement learning strategy is 5 Hz.

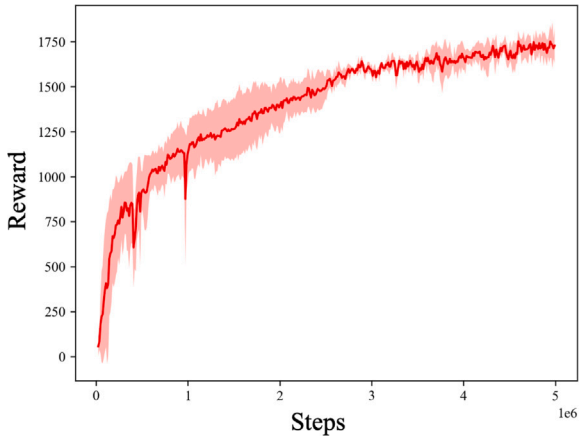


Fig. 10. The training results of RL based on domain randomization. The red solid line and the red shaded area represent the mean and variance, respectively, across 10 repeated experiments with different random seeds.

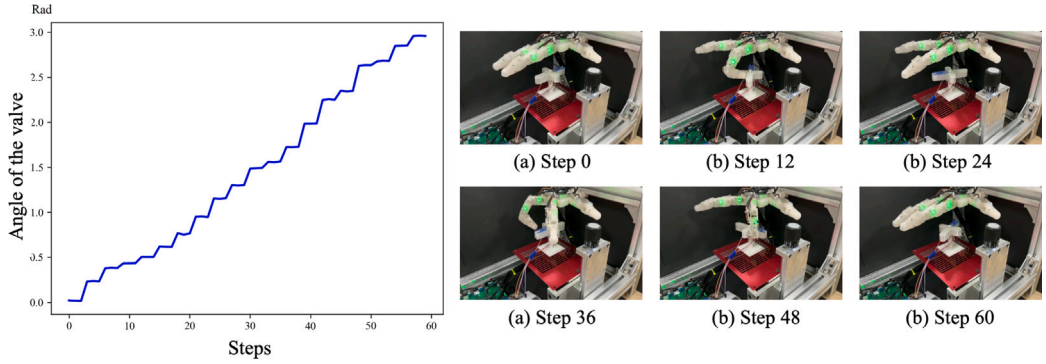


Fig. 11. The constructed real-world environment for valve rotation task. The left image illustrates the change in valve angle over time steps, while the right side shows the actions of the robot and the valve position at different time steps.

Data availability

No data was used for the research described in the article.

References

[1] O.M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al., Learning dexterous in-hand manipulation, Int. J. Robot. Res. 39 (1) (2020) 3–20.

[2] Y. Jin, X. Liu, Y. Shao, H. Wang, W. Yang, High-speed quadrupedal locomotion by imitation-relaxation reinforcement learning, Nat. Mach. Intell. 4 (12) (2022) 1198–1208.

[3] C. Tao, J. Xue, Z. Zhang, Z. Gao, Parallel deep reinforcement learning method for gait control of biped robot, IEEE Trans. Circuits Syst. II: Express Briefs 69 (6) (2022) 2802–2806.

[4] A. Bicchi, Hands for dexterous manipulation and robust grasping: a difficult road toward simplicity, IEEE Trans. Robot. Autom. 16 (6) (2000) 652–662, http://dx.doi.org/10.1109/70.897777.

[5] T.N. Le, J. Lundell, F.J. Abu-Dakka, V. Kyrki, Deformation-aware data-driven grasp synthesis, IEEE Robot. Autom. Lett. 7 (2) (2022) 3038–3045.

[6] E. Mattar, A survey of bio-inspired robotics hands implementation: New directions in dexterous manipulation, Robot. Auton. Syst. 61 (5) (2013) 517–544, http://dx.doi.org/10.1016/j.robot.2012.12.005.

[7] K. Nazarpour, A more human prosthetic hand, Sci. Robot. 5 (46) (2020) eabd9341.

[8] A. Saudabayev, H.A. Varol, Sensors for robotic hands: A survey of state of the art, IEEE Access 3 (2015) 1765–1782.

[9] S. Dhruv, T. Kirti, P. Aviral, S. Krishna, Shadow hand, J. Adv. Res. Appl. Sci. (2014) 04–07.

[10] H. Liu, K. Wu, P. Meusel, N. Seitz, G. Hirzinger, M. Jin, Y. Liu, S. Fan, T. Lan, Z. Chen, Multisensory five-finger dexterous hand: The DLR/HIT hand II, in: 2008 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2008, pp. 3692–3697.

[11] J. Zhou, J. Yi, X. Chen, Z. Liu, Z. Wang, BCL-13: A 13-DOF soft robotic hand for dexterous grasping and in-hand manipulation, IEEE Robot. Autom. Lett. 3 (4) (2018) 3379–3386.

[12] A. Pagoli, F. Chapelle, J.A. Corrales, Y. Mezouar, Y. Lapusta, A soft robotic gripper with an active palm and reconfigurable fingers for fully dexterous in-hand manipulation, IEEE Robot. Autom. Lett. 6 (4) (2021) 7706–7713.

[13] S. Yuan, A.D. Epps, J.B. Nowak, J.K. Salisbury, Design of a roller-based dexterous hand for object grasping and within-hand manipulation, in: 2020 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2020, pp. 8870–8876.

[14] S. Zuo, J. Li, M. Dong, Design, modeling, and manipulability evaluation of a novel four-dof parallel gripper for dexterous in-hand manipulation, J. Mech. Sci. Technol. 35 (2021) 3145–3160.

[15] W. Han, R. Tedrake, Local trajectory stabilization for dexterous manipulation via piecewise affine approximations, in: 2020 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2020, pp. 8884–8891.

[16] M. Pfanne, M. Chalon, F. Stulp, H. Ritter, A. Albu-Schäffer, Object-level impedance control for dexterous in-hand manipulation, IEEE Robot. Autom. Lett. 5 (2) (2020) 2987–2994.

[17] K. Chatzilygeroudis, B. Fichera, I. Lauzana, F. Bu, K. Yao, F. Khadivar, A. Billard, Benchmark for bimanual robotic manipulation of semi-deformable objects, IEEE Robot. Autom. Lett. 5 (2) (2020) 2443–2450.

[18] F. Osiurak, C. Jarry, D. Le Gall, Grasping the affordances, understanding the reasoning: toward a dialectical theory of human tool use, Psychol Rev 117 (2) (2010) 517.

[19] A. Allshire, M. MittaI, V. Lodaya, V. Makoviychuk, D. Makoviichuk, F. Widmaier, M. Wüthrich, S. Bauer, A. Handa, A. Garg, Transferring dexterous manipulation from gpu simulation to a remote real-world trifinger, in: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2022, pp. 11802–11809.

[20] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2017, pp. 23–30.

[21] Z. Deng, J.W. Zhang, Learning synergies based in-hand manipulation with reward shaping, CAAI Trans. Intell. Technol. 5 (3) (2020) 141–149.

[22] P. Ruppel, J. Zhang, Learning object manipulation with dexterous hand-arm systems from human demonstration, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2020, pp. 5417–5424.

[23] T. Chen, J. Xu, P. Agrawal, A system for general in-hand object re-orientation, in: Conference on Robot Learning, PMLR, 2022, pp. 297–307.

[24] R.S. Sutton, A.G. Barto, Reinforcement Learning: An introduction, MIT Press, 2018.

[25] A.Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: Theory and application to reward shaping, in: Icml, vol. 99, Citeseer, 1999, pp. 278–287.

[26] E. Wiewiora, G.W. Cottrell, C. Elkan, Principled methods for advising reinforcement learning agents, in: Proceedings of the 20th International Conference on Machine Learning, ICML-03, 2003, pp. 792–799.

[27] Y. Dong, X. Tang, Y. Yuan, Principled reward shaping for reinforcement learning via lyapunov stability theory, Neurocomputing 393 (2020) 83–90.

[28] Y. Dong, S. Zhang, X. Liu, Y. Zhang, T. Shen, Variance aware reward smoothing for deep reinforcement learning, Neurocomputing 458 (2021) 327–335.

[29] E. Coumans, Y. Bai, Pybullet, a python module for physics simulation for games, robotics and machine learning, 2016–2020, http://pybullet.org.

[30] L. Pan, Q. Cai, L. Huang, Softmax deep double deterministic policy gradients, Adv. Neural Inf. Process. Syst. 33 (2020) 11767–11777.

[31] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.

[32] Z. Deng, X. Liu, Y. Dong, Reward shaping via expectation maximization method, Neurocomputing 609 (2024) 128471.



Zhelin Deng received his bachelor degree from the School of Mathematics and Statistics, Huazhong University of Science and Technology in 2018, and his master degree from the College of Business, City University of Hong Kong, China in 2020. He is currently working towards the Ph.D. degree at School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China. His research interest covers RL in smart manufacturing.



Yunlong Dong received the B.E. degree and Ph.D. degrees from the School of Automation, Huazhong University of Science and Technology, Wuhan, China, in 2017 and 2022. He is currently a postdoctoral researcher in the Department of Automation, Tsinghua University, Beijing, China. His research interests include RL, control theory and robotics.



Xing Liu received the B.E. degree from the School of Electrical and Automation Engineering, East China Jiaotong University in 2020, and his master degree from the School of Information Science and Engineering, East China University of Science and Technology, Shanghai, China. He is currently working towards the Ph.D. degree at School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, China. His research interests include RL, imitation learning and robotics.